



Список тренингов / Промпт-инжиниринг / Промпт-инжиниринг 2.0 / Модуль 8. Дообучение нейросетей (/teach/control/stream/view/id/931478148)

PE8.3 Fine-tuning. Библиотека Unsloth

Предыдущий урок

[\(/pl/teach/control/lesson/view?id=338667547\)](/pl/teach/control/lesson/view?id=338667547)

Следующий урок

[\(/pl/teach/control/lesson/view?id=341126240\)](/pl/teach/control/lesson/view?id=341126240)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)

Уважаемые студенты!

В рамках данного урока могут рассматриваться примеры передачи персональных данных при регистрации на иностранных сервисах, а также обработки и хранения пользовательских данных.

В соответствии с требованиями Федерального закона № 152-ФЗ «О персональных данных», обращаем ваше внимание: в демонстрационных примерах и при выполнении домашних заданий необходимо использовать исключительно тестовые
Действия

данные или тестовые аккаунты, в которых не указаны персональные данные — используйте вымышленные ники, дополнительные данные и т.п.

Использование иностранных сервисов не является обязательным. Вы вправе выполнять домашние задания на аналогичных разрешённых платформах по вашему выбору.

Сегодня мы углубимся в Fine-tuning и разберём реальный пример дообучения нейросети для создания чат-бота. Ознакомьтесь с уроком и внимательно изучите блокнот эксперта. В этом уроке домашнего задания для практики не предусмотрено.

Результат дня

Рассмотрим на практике, как работает fine-tuning языковых моделей.

Теория на сегодня

- **Разберём** процесс fine-tuning языковых моделей.
- **Изучим** библиотеку Unsloth и её преимущества.
- **Разберём на практике** ноутбук с оптимизированным обучением.

Время изучения материалов

 Время чтения ~ 40
и просмотра: мин

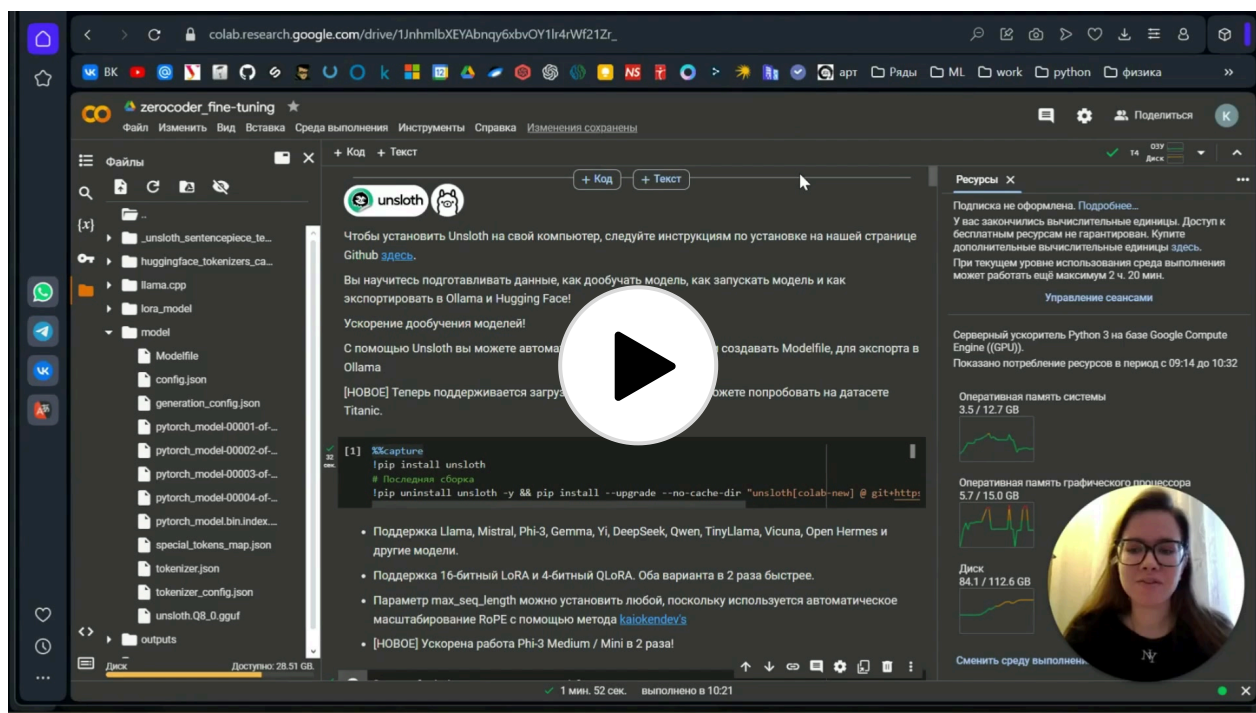
Эксперт

Маяцкая Екатерина

• Научный сотрудник в Уральском Федеральном Университете

- Обработка естественного языка, работа с временными рядами
- Исследовательские проекты со Сбером
- Участие в международных конференциях, публикации с индексацией в Scopus

1. Введение

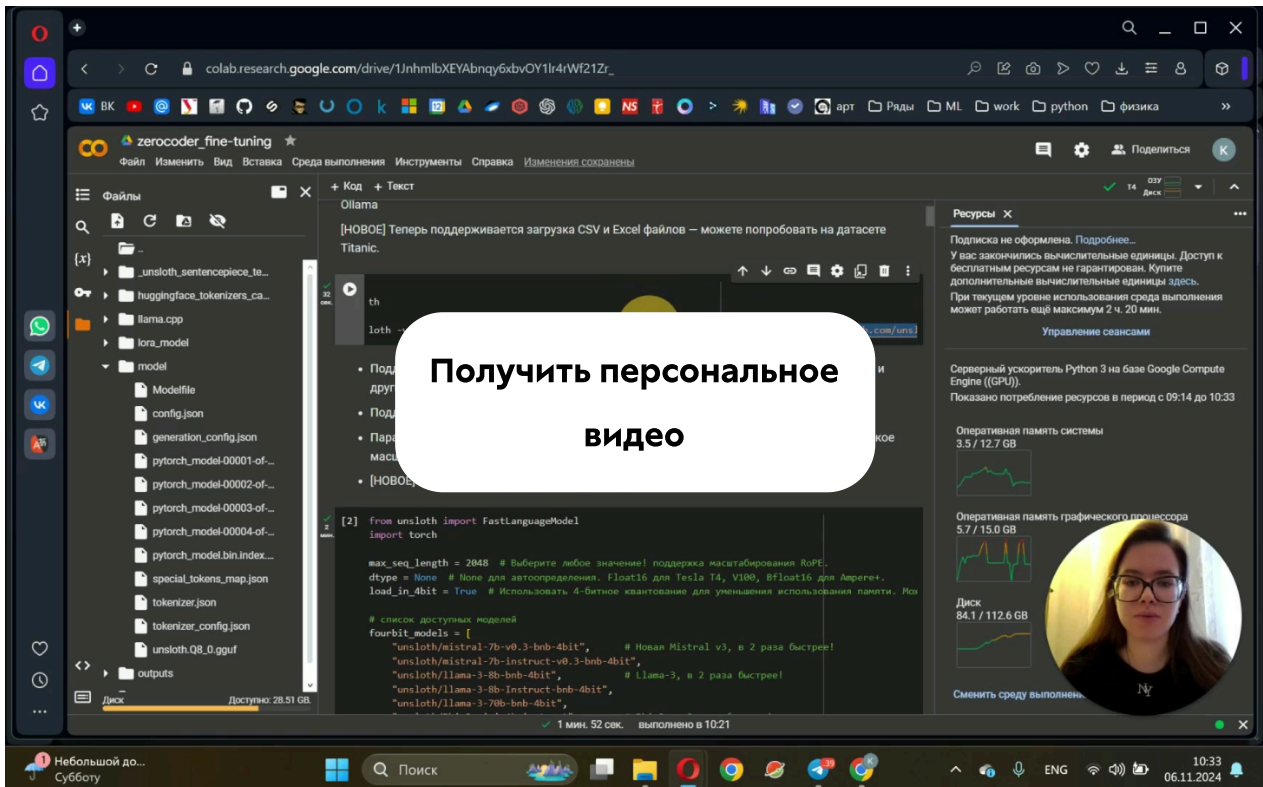


Сегодня мы обсудим процесс fine-tuning (дальнейшего обучения) больших языковых моделей и детально разберём соответствующий ноутбук. Основной инструмент, который мы будем использовать — это библиотека **Unsloth**. Она поможет оптимизировать процесс обучения, а также снизить требования к вычислительным ресурсам, что делает работу с большими моделями проще и эффективнее.

Библиотека Unsloth предлагает несколько ключевых функций:

- экспорт данных в популярные платформы **Ollama** и **Hugging Face**;
- автоматическое создание файла модулей, что облегчает экспорт обученной модели в **Ollama**;
- поддержка квантования и работа с моделями различных размеров и конфигураций, что ускоряет обучение и оптимизирует использование памяти.

2. Подготовка ноутбука к дообучению



💡 Ноутбук эксперта с комментариями —
https://colab.research.google.com/drive/1VRj3YB7A6_ahw5NUHBqnxqNuOaAdSSmR?usp=sharing
(https://colab.research.google.com/drive/1VRj3YB7A6_ahw5NUHBqnxqNuOaAdSSmR?usp=sharing)

Информация о библиотеке Unsloth —
[https://github.com/unslothai/unsloth#installation-instructions---](https://github.com/unslothai/unsloth#installation-instructions---conda)
conda ([https://github.com/unslothai/unsloth#installation-instructions---](https://github.com/unslothai/unsloth#installation-instructions---conda)
--conda)

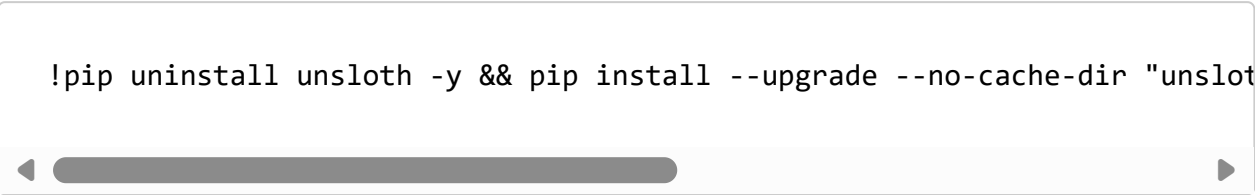
Для начала работы с библиотекой Unsloth установим её с помощью команды:

```
!pip install unsloth
```

Действия

Установка с официального репозитория GitHub позволяет гарантированно получать последнюю версию с актуальными обновлениями.

```
!pip uninstall unsloth -y && pip install --upgrade --no-cache-dir "unsloth
```

A terminal window with a dark background. The command '!pip uninstall unsloth -y && pip install --upgrade --no-cache-dir "unsloth' is visible. Below the command is a horizontal scrollbar with a grey track and a white slider.

Unsloth поддерживает множество моделей, таких как **Llama**, **Mistral**, **GEMMA**, и другие. Полный список моделей доступен на их официальном репозитории GitHub и на Hugging Face. Кроме того, Unsloth позволяет работать с квантованием моделей (например, 16-битное и 4-битное квантование), что существенно ускоряет обучение.

Для обучения мы устанавливаем **max sequence length** — параметр, ограничивающий максимальную длину текста, который будет обрабатываться. В Unsloth реализован метод `rule`, позволяющий работать с длинными последовательностями, что делает библиотеку более гибкой.

```

from unsloth import FastLanguageModel
import torch

max_seq_length = 2048 # Выберите любое значение! поддержка масштабирования
dtype = None # None для автоопределения. Float16 для Tesla T4, V100, Bf16
load_in_4bit = True # Использовать 4-битное квантование для уменьшения и

# список доступных моделей
fourbit_models = [
    "unsloth/mistral-7b-v0.3-bnb-4bit",      # Новая Mistral v3, в 2 раза
    "unsloth/mistral-7b-instruct-v0.3-bnb-4bit",
    "unsloth/llama-3-8b-bnb-4bit",          # Llama-3, в 2 раза быстрее!
    "unsloth/llama-3-8b-Instruct-bnb-4bit",
    "unsloth/llama-3-70b-bnb-4bit",
    "unsloth/Phi-3-mini-4k-instruct",       # Phi-3, в 2 раза быстрее!
    "unsloth/Phi-3-medium-4k-instruct",
    "unsloth/mistral-7b-bnb-4bit",
    "unsloth/gemma-7b-bnb-4bit",           # Gemma, в 2.2 раза быстрее!
] # Больше моделей можно найти на https://huggingface.co/unsloth

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/llama-3-8b-bnb-4bit",
    max_seq_length = max_seq_length,
    dtype = dtype,
    load_in_4bit = load_in_4bit,
    # token = "hf_...", # Используйте токен, если работаете с моделями, к
)

```

При инициализации модели задаём параметры:

- **dtype:** по умолчанию используется `None`, что позволяет системе автоматически определять тип данных;
- **Load in 4-bit:** Устанавливаем этот параметр в `True`, чтобы использовать 4-битное квантование и сэкономить вычислительные ресурсы.

Для загрузки моделей используем метод `from_pretrained`, разработанный Unsloth. Он не только загружает модель, но и учитывает все параметры оптимизации, что упрощает процесс fine-tuning.

tuning.

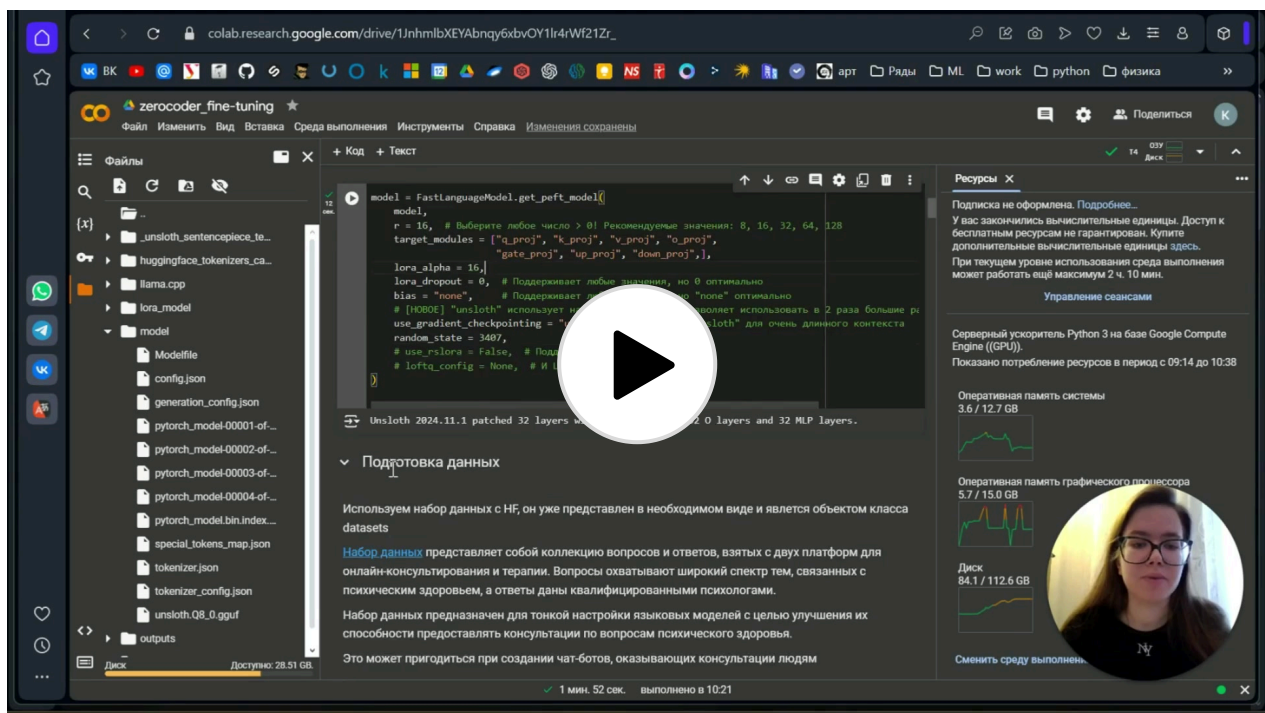
💡 <https://huggingface.co/unsloth> (<https://huggingface.co/unsloth>) — другие модели и блокноты к ним.

```
model = FastLanguageModel.get_peft_model(
    model,
    r = 16, # Выберите любое число > 0! Рекомендуемые значения: 8, 16, 32
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj",],
    lora_alpha = 16,
    lora_dropout = 0, # Поддерживает любые значения, но 0 оптимально
    bias = "none",    # Поддерживает любые значения, но "none" оптимально
    # [НОВОЕ] "unsloth" использует на 30% меньше VRAM, позволяет использовать
    use_gradient_checkpointing = "unsloth", # True или "unsloth" для checkpointing
    random_state = 3407,
    # use_rslora = False, # Поддержка rank stabilized LoRA
    # loftq_config = None, # И LoftQ
)
```

Для настройки обучения моделей с использованием Lora (Low-Rank Adaptation), используем функцию `get_peft_model`. Эта настройка позволяет обновлять только часть весов, что экономит ресурсы:

- `r = 16`: уровень низкого ранга, который влияет на количество изменяемых параметров;
- `target_modules`: список весовых матриц, которые мы будем обновлять в процессе обучения;
- `lora_alpha`: уровень активности параметров Lora;
- `bias`: смещение;
- `use_gradient_checkpointing`: опция, позволяющая оптимизировать обработку длинных текстов;
- `random_state`: фиксация результатов.

3. Подготовка данных



Для обучения мы используем датасет вопросов и ответов с платформы Hugging Face, посвященный темам психологического здоровья и терапии (https://huggingface.co/datasets/Amod/mental_health_counseling_conversations?row=1 (https://huggingface.co/datasets/Amod/mental_health_counseling_conversations?row=1)). Этот датасет подходит для создания чат-ботов, например, для консультаций.

1. Мы загружаем его с помощью команды:

```
from datasets import load_dataset
dataset = load_dataset("Amod/mental_health_counseling_conversations", split="train")
print(dataset.column_names)
```

2. Для разделения данных на тренировочные и тестовые используем `split = "train"`, выделяя 20% данных для тестирования.

Действия


```
train_test_split = dataset.train_test_split(test_size=0.2) # 20% данных

train_dataset = train_test_split['train']
test_dataset = train_test_split['test']

print("Train dataset size:", len(train_dataset))
print("Test dataset size:", len(test_dataset))
```

После запуска программы мы можем увидеть количество примеров в каждом датасете.

3. Стандартизируем данные и подаём датасет:

```
from unsloth import standardize_sharegpt
dataset = standardize_sharegpt(dataset)
```

4. Может возникнуть проблема, которая заключается в том, что некоторые наборы данных содержат несколько столбцов. Для того чтобы Ollama и llama.cpp работали как чат-бот ChatGPT, необходимо, чтобы в наборе данных было только два столбца — instruction (инструкция) и output (выход). Для решения этой задачи мы сделаем следующее:

Объединим все столбцы в один prompt.

Используем функцию `to_sharegpt`, чтобы выполнить процесс слияния столбцов!

```

from unsloth import to_sharegpt
dataset = to_sharegpt(
    dataset,
    merged_prompt = "{Context}",
    output_column_name = "Response",
    # conversation_extension = 1,

```

Функция **conversation_extension** позволяет создать фейковый набор данных с несколькими сообщениями в диалоге, в контексте чата. Числовое значение указывает, сколькими примерами продолжится диалог.

5. Чтобы модель могла работать в формате "вопрос-ответ", преобразуем данные в нужный формат. Метод **apply_chat_template** применяет шаблон к данным.

```

chat_template = """Below are some conversations. Write responses that app

### Question:
{INPUT}

### Corresponding answer:
{OUTPUT}"""

from unsloth import apply_chat_template
dataset = apply_chat_template(
    dataset,
    tokenizer = tokenizer,
    chat_template = chat_template,
    # default_system_message = "You are a helpful assistant", << [OPTIONAL

```

6. Модель ещё не дообучена. Посмотрим, как она отвечает. Метод **FastLanguageModel.for_inference** оптимизирует работу модели и ускоряет генерацию текста. Мы передаём запрос с описанием симптомов. Снова используем **apply_chat_template** и токенизацию.

```

import time

FastLanguageModel.for_inference(model) # Ускорение модели
messages = [
    {"role": "user", "content": "I'm going through some things with my fe
]
input_ids = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt = True,
    return_tensors = "pt",
).to("cuda")

# Время начала генерации
start_time = time.time()
# Генерируем текст
output_ids = model.generate(input_ids, max_new_tokens=100, pad_token_id=t

# Время окончания генерации
end_time = time.time()

# Время, затраченное на генерацию
generation_time = end_time - start_time
print(f"Время генерации: {generation_time:.2f} секунд")

```

7. Ответ получен в виде токенов. Декодируем токены в текст, получаем результат.

```

output_text = tokenizer.decode(output_ids[0], skip_special_tokens=True)
print(output_text)

```

8. Изучаем токенизированный ответ.

```

print(output_ids)
Действия

```

9. Изучаем токенизированный датасет.

```
print(dataset['text'][1]) # Покажет первые 10 элементов из столбца 'text'
```

4. Дообучение

Получить персональное видео

1. Используем **SFTTrainer** из библиотеки Huggingface.
SFTTrainer нужен для обучения на размеченных данных.

```

from trl import SFTTrainer
from transformers import TrainingArguments
from unsloth import is_bfloat16_supported
trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    # compute_metrics=compute_metrics,
    train_dataset = dataset, # тренировочный датасет
    dataset_text_field = "text", # указываем поле, на котором обучается модель
    max_seq_length = max_seq_length,
    dataset_num_proc = 2, # количество процессов для параллельной обработки
    packing = False, # короткие данные не будут упаковываться в длинные
    args = TrainingArguments( # задаём гиперпараметры
        per_device_train_batch_size = 2, # задаём размер банчей
        gradient_accumulation_steps = 4, # количество шагов, после которых делаем шаг оптимизации
        warmup_steps = 2, # количество "разогревочных" шагов
        max_steps = 60, # максимальное количество шагов до конца обучения
        # num_train_epochs = 3, # Для длительного дообучения
        learning_rate = 0.001, # скорость обучения
        fp16 = not is_bfloat16_supported(), # использование 16-битных чисел
        bf16 = is_bfloat16_supported(),
        logging_steps = 1, # частота вывода логов на протяжении обучения
        optim = "adamw_8bit", # оптимизатор для экономии памяти
        weight_decay = 0.01, # уменьшение переобучения, уменьшение весов
        lr_scheduler_type = "linear", # тип планировщика
        seed = 3407,
        output_dir = "outputs", # куда сохранятся все результаты обучения
    ),
)

```

2. Выводим параметры системы. Отключаем доступ к wandb.

```

import wandb
# Отключение логирования wandb
wandb.init(mode='disabled')

```

3. Передаём метод **train** для обучения. Происходит обучение.

```
trainer_stats = trainer.train()
```

4. Смотрим статистику системы после обучения.

```
#@title Показать итоговую статистику по расходу памяти и времени
used_memory = round(torch.cuda.max_memory_reserved() / 1024 / 1024 / 1024, 3)
used_memory_for_lora = round(used_memory - start_gpu_memory, 3)
used_percentage = round(used_memory / max_memory * 100, 3)
lora_percentage = round(used_memory_for_lora / max_memory * 100, 3)
print(f"{trainer_stats.metrics['train_runtime']} seconds used for training")
print(f"{round(trainer_stats.metrics['train_runtime']/60, 2)} minutes used for training")
print(f"Peak reserved memory = {used_memory} GB.")
print(f"Peak reserved memory for training = {used_memory_for_lora} GB.")
print(f"Peak reserved memory % of max memory = {used_percentage} %.")
print(f"Peak reserved memory for training % of max memory = {lora_percentage} %.")
```

5. Выводим инференс модели — как модель генерирует текст после обучения. Используем тот же самый промпт, что и раньше.

```

FastLanguageModel.for_inference(model)
messages = [
    {"role": "user", "content": "I'm going through some things with my fe
]
input_ids = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt = True,
    return_tensors = "pt",
).to("cuda")

# Время начала генерации
start_time = time.time()
# Генерируем текст
output_ids = model.generate(input_ids, max_new_tokens=150, pad_token_id=t

# Время окончания генерации
end_time = time.time()

# Время, затраченное на генерацию
generation_time = end_time - start_time
print(f"Время генерации: {generation_time:.2f} секунд")

```

6. Сохраняем модели локально. Сохраняем модель LoRA, токенайзер.

```

model.save_pretrained("lora_model") # Local saving
tokenizer.save_pretrained("lora_model")
# model.push_to_hub("your_name/lora_model", token = "...") # Online saving
# tokenizer.push_to_hub("your_name/lora_model", token = "...") # Online saving

```

7. Загружаем и используем сохранённые модели.

```

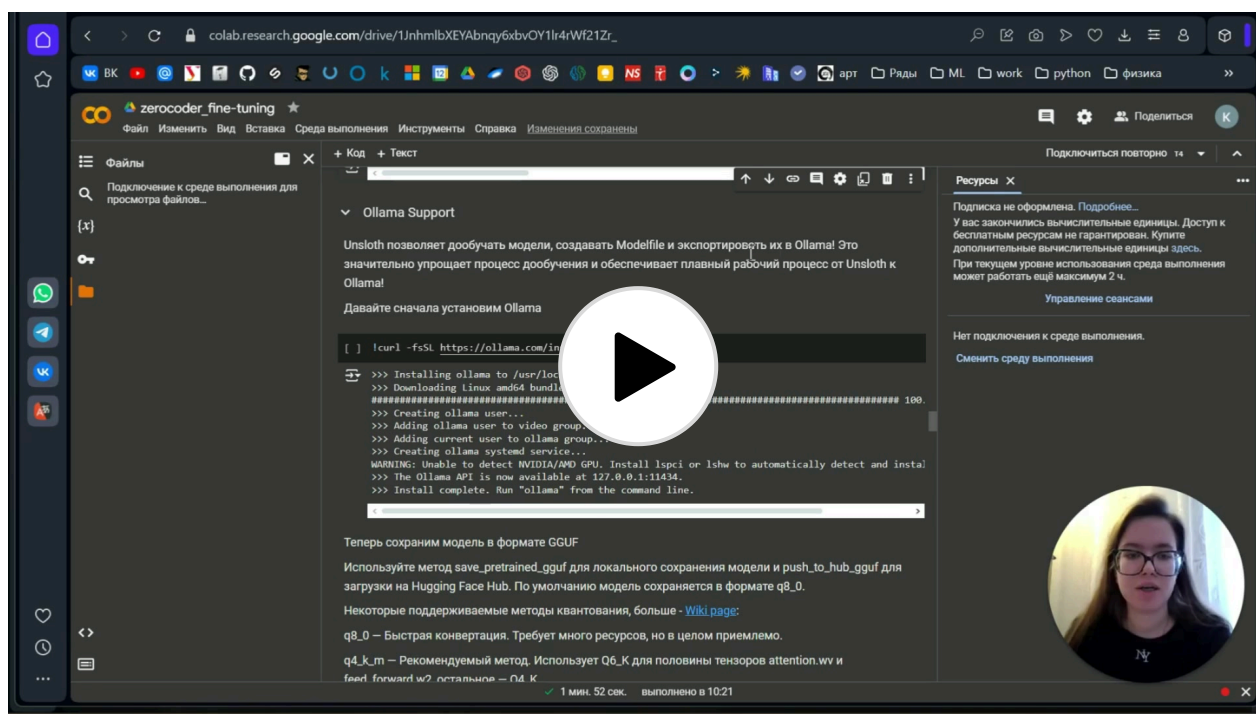
if False:
    from unsloth import FastLanguageModel
    model, tokenizer = FastLanguageModel.from_pretrained(
        model_name = "lora_model", # ваша дообученная модель
        max_seq_length = max_seq_length,
        dtype = dtype,
        load_in_4bit = load_in_4bit,
    )
    FastLanguageModel.for_inference(model)
pass

messages = [
    {"role": "user", "content": "Describe anything special about a sequer
]
input_ids = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt = True,
    return_tensors = "pt",
).to("cuda")

from transformers import TextStreamer
text_streamer = TextStreamer(tokenizer, skip_prompt = True)
_ = model.generate(input_ids, streamer = text_streamer, max_new_tokens =

```

5. Ollama



1. Сохраняем модели на Ollama. Для этого устанавливаем Ollama.

```
!curl -fsSL https://ollama.com/install.sh | sh
```

2. Сохраняем модель в файл GGUF.

Используем метод `save_pretrained_gguf` для локального сохранения модели и `push_to_hub_gguf` для загрузки на Hugging Face Hub. По умолчанию модель сохраняется в формате `q8_0`.

```

# Save to 8bit Q8_0
if True: model.save_pretrained_gguf("model", tokenizer,)
# За HF токеном идем - https://huggingface.co/settings/tokens
# Измени hf на свое имя!
# Создай в своем профиле на HF модель с именем model , чтобы сохранить в
if False: model.push_to_hub_gguf("hf/model", tokenizer, token = "")

# Сохранить в 16bit GGUF
if False: model.save_pretrained_gguf("model", tokenizer, quantization_met
if False: model.push_to_hub_gguf("hf/model", tokenizer, quantization_meth

# Сохранить в q4_k_m GGUF
if False: model.save_pretrained_gguf("model", tokenizer, quantization_met
if False: model.push_to_hub_gguf("hf/model", tokenizer, quantization_meth

# Сохранить в различные форматы GGUF
if False:
    model.push_to_hub_gguf(
        "hf/model", # Измени hf на своем имя!
        tokenizer,
        quantization_method = ["q4_k_m", "q8_0", "q5_k_m",],
        token = "", # За HF токеном идем - https://huggingface.co/settin

```

Обычно используется либо локальное сохранение, либо сохранение на Ollama и HF.

3. Сохраняем модели на HF. Указываем свой аккаунт и токен.

```

if True: model.push_to_hub_gguf("аккаунт/model", tokenizer, token = "токе

```

4. Подключаемся к серверу Ollama через API с использованием Python.

Действия

```
!sudo apt-get install -y pciutils
!curl -fsSL https://ollama.com/install.sh | sh
from IPython.display import clear_output

import os
import threading
import subprocess
import requests
import json

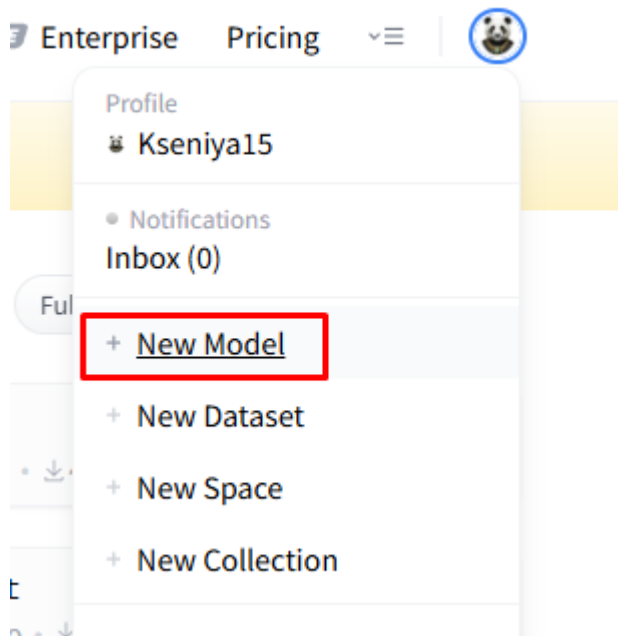
def ollama():
    os.environ['OLLAMA_HOST'] = '0.0.0.0:11434'
    os.environ['OLLAMA_ORIGINS'] = '*'
    subprocess.Popen(["ollama", "serve"])

ollama_thread = threading.Thread(target=ollama)
ollama_thread.start()
```

5. Загружаем модель с HF. Указываем путь.

```
!ollama pull hf.co/katya1836/model
```

Чтобы создать модель на Hugging Face, перейди на сайт <https://huggingface.co/> (<https://huggingface.co/>), нажми на свой профиль, выбери **New model** и назови её **model**.



Create a new model repository

A repository contains all model files, including the revision history.

Owner

Kseniya15

Model name

model

License

License

Base template



Public

Anyone on the internet can see this model. Only you (personal model) or members of your organization (organization model) can commit.



Private

Only you (personal model) or members of your organization (organization model) can see and commit to this model.

Once your model is created, you can upload your files using the web interface or git.

Create model

6. Работаем с моделью.
Действия

```
prompt="""
### Question:
I'm going through some things with my feelings and myself. I barely sleep

### Corresponding answer:
"""

response = requests.post(
    'http://localhost:11434/api/generate',
    json={
        'model': "hf.co/katya1836/model",
        "options": { "temperature": 0, "num_predict": 150},
        "prompt": prompt,
        "stream": False
    }
)

# Декодируем содержимое ответа
result = response.content.decode('utf-8')
result_json = json.loads(result)
# Извлекаем текст из ключа 'response'
generated_text = result_json.get('response')
print(generated_text)
```

7. Загружаем модель на Ollama.

```
!sudo apt-get install -y pciutils
!curl -fsSL https://ollama.com/install.sh | sh
from IPython.display import clear_output

import os
import threading
import subprocess
import requests
import json

def ollama():
    os.environ['OLLAMA_HOST'] = '0.0.0.0:11434'
    os.environ['OLLAMA_ORIGINS'] = '*'
    subprocess.Popen(["ollama", "serve"])

ollama_thread = threading.Thread(target=ollama)
ollama_thread.start()
```

8. Выводим modelfile — файл, который содержит конструкции промпта.

```
print(tokenizer._ollama_modelfile)
```

9. Создаём модель на Ollama. Нужно будет указать путь к modelfile. Модель создаётся на Ollama.

```
!ollama create unsloth_model -f ./model/Modelfile
```

10. Запускаем модель с Ollama в Терминале:

```
!ollama run unsloth_model
```

Действия

Каждый раз использование модели с Ollama происходит так:

Подключение к серверу Ollama — создание модели — работа с моделью.

```
prompt="""
### Question:
I'm going through some things with my feelings and myself. I barely sleep

### Corresponding answer:
"""

response = requests.post(
    'http://localhost:11434/api/generate',
    json={
        'model': "unsloth_model",
        "options": { "temperature": 0, "num_predict": 150},
        "prompt": prompt,
        "stream": False
    }
)

# Декодируем содержимое ответа
result = response.content.decode('utf-8')
result_json = json.loads(result)
# Извлекаем текст из ключа 'response'
generated_text = result_json.get('response')
print(generated_text)
```

Для загрузки модели на Ollama нам нужно сперва загрузить модель локально. При использовании HF модель находится на HF, и локальное сохранение не нужно.

11. Имитируем работу модели через командную строку.

```
from IPython.display import JSON
from google.colab import output
from subprocess import getoutput
import os

def shell(command):
    if command.startswith('cd'):
        path = command.strip().split(maxsplit=1)[1]
        os.chdir(path)
        return JSON([''])
    return JSON([getoutput(command)])
output.register_callback('shell', shell)
```

Подведение итогов

- узнали, как дообучать модель;
- узнали о библиотеке Unsloth, которая позволяет ускорить обучение;
- научились загружать модель и использовать.

Созданные модели можно адаптировать к разным задачам.

Важно экспериментировать с параметрами, которые влияют на работу и обучение модели.

На следующем занятии

Перейдём к изучению нового модуля на тему **"Оптимизация затрат на ИИ: стратегии развертывания ИИ-моделей"**.

Если у вас есть вопросы, задавайте их, пожалуйста, в учебном чате, а не в этой анкете, — так вы сможете быстрее получить ответ.

Урок

Действия

Понравился ли урок?

- ☐ Понравился, все было понятно
- ☒ Неплохо, но не все получилось/не все понятно
- ☐ Не понравился

Обратная связь по уроку (не обязательно)

Отправить

ЗАДАНИЕ



☐ Я посмотрел(-а) урок. - *обязательное поле

Отправить ответ

Сохранить как черновик

20 монет за выполнение этого урока

Предыдущий
урок

(<https://university.zerocoder.ru/pl/teach/control/lesson/view?id=338667547>)

Следующий урок